

Technology Stack

Date	15 March 2026
Team ID	
Project Name	Smart Agricultural Production Optimization Engine (OptiCrop)
Maximum Marks	2 Marks

Define Your Technology Stack:

Identify the programming languages, frameworks, databases, front-end tools, back-end tools, and APIs that you will use to build your solution. A well-chosen technology stack ensures that your application is scalable, maintainable, and aligned with your project requirements.

Fill out the table below to detail the specific technologies chosen for each component of your architecture and provide a brief justification for your choice.

The Technology Stack defines the set of technologies, tools, frameworks, databases, and APIs used to build the Smart Agricultural Production Optimization Engine (OptiCrop).

Selecting the right technology stack is important for ensuring scalability, performance, security, and maintainability of the system.

For this project, modern web technologies are used for frontend development to create a responsive and user-friendly interface for farmers and agricultural stakeholders.

The backend is developed using Python and Django to handle data processing, business logic, and machine learning integration efficiently.

A robust database system is used to store agricultural datasets such as soil nutrient values (N, P, K), temperature, humidity, pH, rainfall, crop details, and prediction results. Cloud deployment technologies provide reliable hosting and scalability for large-scale usage.

S.No	Architecture Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side (e.g., React, Angular; HTML/CSS)	React.js, HTML5, CSS3, Bootstrap	Provides fast, responsive and interactive user interface. Components are reusable and easy to maintain. Bootstrap ensures a mobile-friendly design for farmers using different devices.
2	Backend / Server-Side (e.g., Node.js, Python/Django, Java)	Python (Django Framework)	Django offers a secure, scalable and robust backend. It has built-in authentication, admin panel and simplifies API development for ML integration.
3	Database / Data Storage (e.g., MySQL, MongoDB, PostgreSQL)	PostgreSQL	PostgreSQL handles structured data (Sensor data, user data, crop details) with strong relational integrity, ACID compliance and excellent performance.
4	Cloud / Hosting / Deployment (e.g., AWS, Azure, Heroku, Docker)	AWS (EC2, S3, RDS), Docker	AWS provides reliable hosting, scalability and security. S3 is used for storage, RDS for database and EC2 for deployment. Docker ensures easy deployment and consistency across environments.
5	Version Control & CI/CD (e.g., GitHub, GitLab, Jenkins)	GitHub, GitHub Actions	GitHub is used for version control and team collaboration. GitHub Actions enables automated testing, building and deployment (CI/CD pipeline).
6	Third-Party APIs / Other Tools (e.g., Stripe, Twilio, Google Maps)	OpenWeatherMap API, Google Maps API, SMTP (Email Service), Pandas, NumPy, Scikit-learn	• OpenWeatherMap API for real-time weather data. • Google Maps API for location services. • SMTP for sending email notifications/alerts. • Pandas & NumPy for data processing and analysis. • Scikit-learn for ML model training and prediction.